

Algorithmic Analysis of Water Distribution Networks

TC2038 Situation Problem 2: all seven tasks

Santiago Hernández Senosiain

November 2, 2026

Outline

Problem and datasets

Tasks and techniques

Findings

Connections

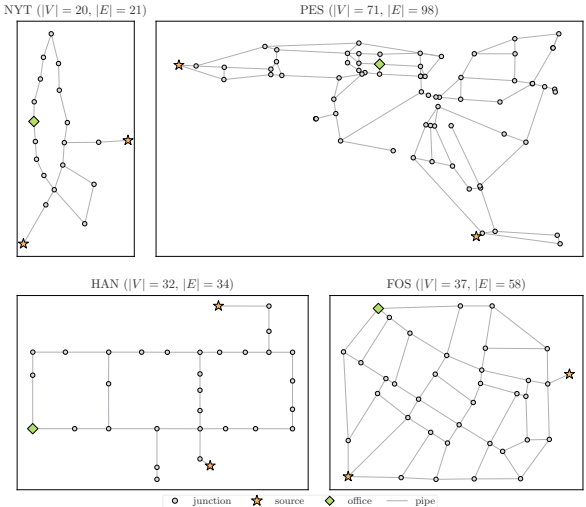
Conclusions

The problem

- Network $G = (V, E)$: junctions with coordinates, two sources, pipes with a capacity
- Pipe length $\ell(u, v) = \|p_u - p_v\|$; network distance d_G under ℓ
- Seven questions, one study: each answer feeds the next

T1	how to store and explore the network
T2	how long the pipes are, how indirect the network is
T3	which source supplies each node, which pipes to close
T4	which node of each sector gets the oldest water
T5	how much water reaches that node, and what limits it
T6	shortest closed sampling walk from the office
T7	where a new node connects

Datasets



- NYT, HAN, FOS, PES: 20 to 71 nodes, 21 to 98 pipes
- All connected, two sources each, very sparse (density ≤ 0.11)
- NYT, HAN near-trees; FOS, PES meshed
- 6 of 8 sources are leaves: one pipe out
- Data issues: PES pipe 28-30 listed twice (kept once); NYT new point $(6478, 3050)$ is a copy of HAN's (rejected)

Tasks and techniques

Task	Technique	Main algorithm	Compared against
T1 representation	graph traversal	BFS, DFS, components	list vs matrix
T2 pipe lengths	geometry, DP	lengths, Floyd-Warshall	scan, hash, sorted
T3 sectorization	greedy	multi-source Dijkstra	Dijkstra per source
T4 freshness	greedy	shortest-path tree	before vs after sectors
T5 maximum flow	network flow	Dinic	Edmonds-Karp
T6 sampling route	greedy, local search, B&B	NN, NN + 2-opt, B&B	exact optimum
T7 expansion	divide and conquer	2-d tree	brute force

- All algorithms hand-written; NetworkX and SciPy only to cross-check them

Three reductions that make the tasks tractable

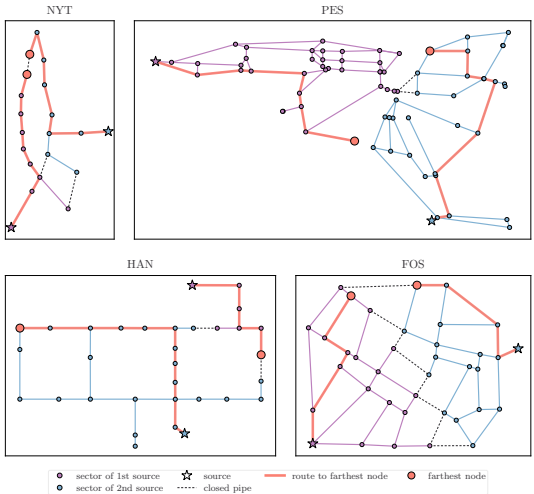
- **T3:** multi-source Dijkstra = Dijkstra from a virtual super-source; first source to settle a node owns it. One run instead of $|S|$
- **T5:** each pipe $\rightarrow$ two arcs of capacity c; max flow = capacity of the min cut, whose saturated pipes are the bottleneck
- **T6:** optimal walk = optimal tour on the metric closure $D_{uv} = d_G(u, v)$, expanded back into real pipes. Branch and bound prunes with

$$\mathrm{LB} = \mathrm{cost} + \mathrm{MST}(R) + \min_{r \in R} D_{ur} + \min_{r \in R} D_{ro}$$

T1 and T2: representation and lengths

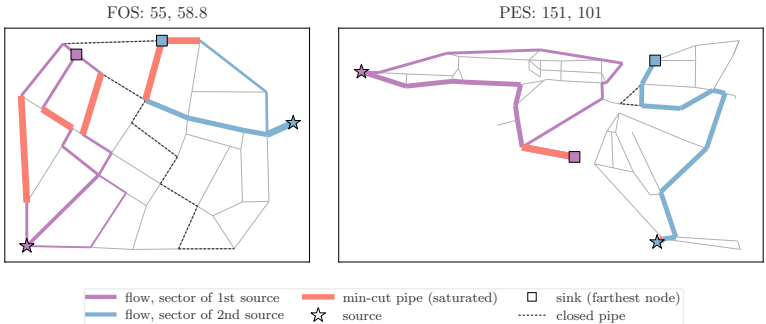
- Matrix BFS 8 to more than 20 times slower than the list; matrix memory twice the list on PES
- Matrix only wins the pipe-existence query: used only for all-pairs tables
- Length and capacity are distinct attributes (correlation -0.65 to $+0.13$)
- Network distance is 19% (FOS) to 83% (PES) longer than straight-line on average
- PES: nodes 19 and 39 are 147 apart, 4,146 by pipe (two clusters, few links)

T3 and T4: sectors and the oldest water



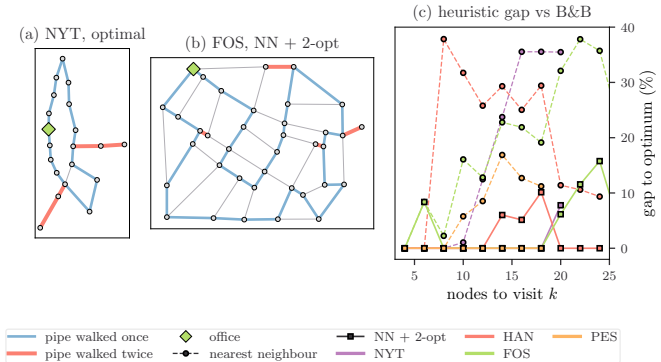
- Pipes to close: 3 (NYT), 2 (HAN), 8 (FOS), 2 (PES)
- Multi-source: half the relaxations of per-source Dijkstra
- 4 of 8 farthest nodes sit at a closed pipe
- Sectors cut a source's worst water age by up to 2.75 times (HAN 13)

T5: shortest route is not maximum flow



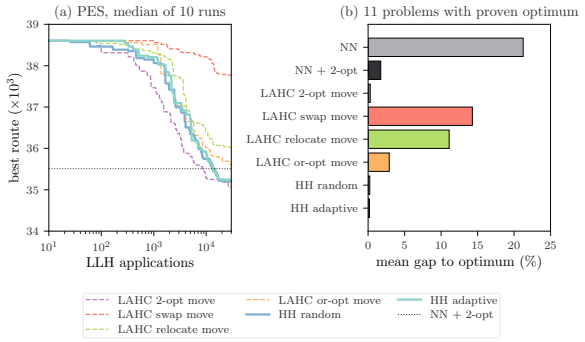
- 6 of 8 sectors: the min cut is the single pipe at a leaf source or sink
- FOS: three bottleneck pipes carry 55, the shortest route alone 19; PES 1: 151 vs 101
- Unlimited cut pipes: no gain in HAN (chains of 16), up to 3.6 times in FOS

T6: sampling routes



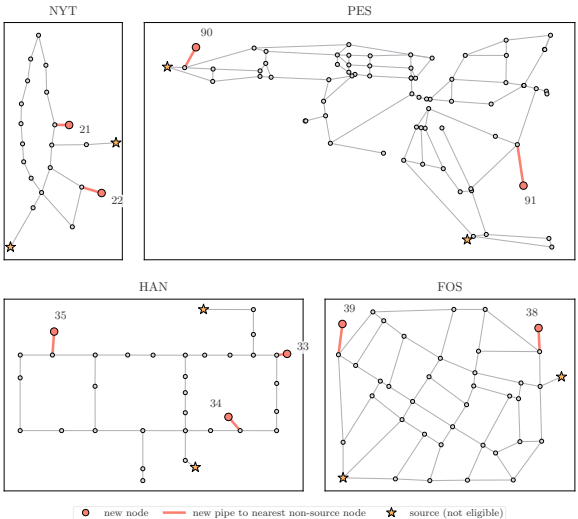
- NYT optimal (7,072, proven by branch and bound); HAN, FOS, PES approximate
- 40 subsets with known optimum: NN 16.0% above, NN + 2-opt 1.9%, optimal in 31

T6 extension: a selection hyper-heuristic



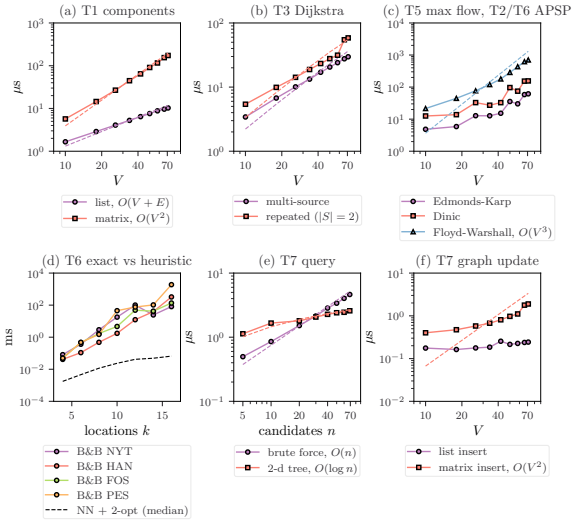
- Each step picks one random move (2-opt, swap, relocate, or-opt); late acceptance (LAHC); 30,000 steps, 10 seeds
- 0.2% mean gap on 11 solved problems vs 0.29% for the best single move (LAHC 2-opt move); best HAN and FOS route in 9 and 6 of 10 runs (vs 3 and 2)
- Best PES route of the study: 34,700, 2.3% below the T6 route

T7: expansion



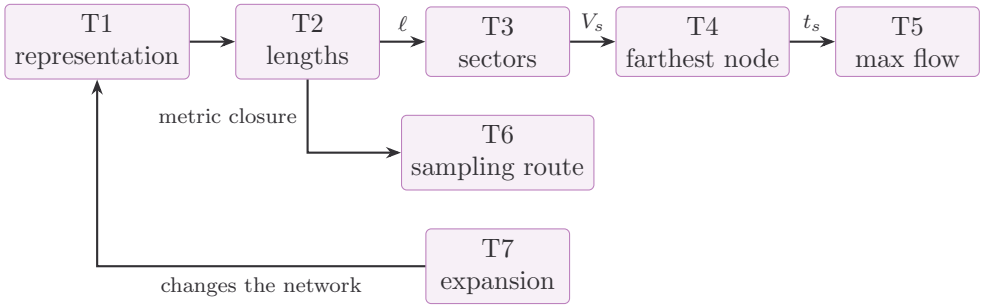
- Each new node links to its nearest non-source node
- FOS point $(7071, 9055)$ becomes node 38 linked to node 17, as in the challenge slides
- 2-d tree checks 6.7 of 18 candidates (NYT) and 11 of 69 (PES) per query
- List update writes 3 entries; matrix copies the whole table

Performance against theory



- List $\sim V + E$, matrix $\sim V^2$, Dijkstra $\sim (V + E) \log V$
- Floyd-Warshall below V^3: NumPy overhead at $V \leq 71$
- Flows far below worst case: at most 4 augmentations
- Branch and bound: about 1.4 to 2.2 times per added location

How the tasks inform each other



- Leaf sources explain the there-and-back legs (T6) and five of the six single-pipe bottlenecks (T5)
- PES's two clusters (T2) explain why two closures separate its sectors (T3)
- Same source-sink pairs: shortest route (T4) vs narrowest cut (T5)

Conclusions and limitations

- T6 is the only hard problem: exponential exact search, optimum proven only for NYT and subsets up to 26 nodes
- Cheap heuristics carry most of the value: NN + 2-opt 1.9% above the optimum on average; the hyper-heuristic adds consistency
- Most expensive polynomial step: Floyd-Warshall, $O(V^3)$; everything else is near linear here

Limitations

- No hydraulics: no pressure, head loss or demand; capacities without units; age $\propto$ distance
- One source-sink pair per sector; stated conventions for duplicates, outliers and insertion order
- Weak bound on near-trees; small instances, so growth measured on subsets

Thanks

Questions?